

PLATFORM ENGINEER

반복되는 운영 절차를 줄이고, 실제 결과로 성공을 확인합니다.

배포와 인프라 변경을 재사용 가능한 흐름으로 만들고, 요청 접수가 아니라 서비스와 자원의 실제 상태를 기준으로 완료 여부를 판단합니다.

- 01

Heimdall

저장소 등록부터 검증된 Preview 전환까지 반복 배포 절차를 자동화

DEPLOYMENT
- 02

Gjallar

Proxmox 변경을 승인·중복 방지·사후 확인으로 통제하는 운영 도구

INFRASTRUCTURE
- 03

K-Le-PaaS

자연어 요청을 제한된 Kubernetes 실행 계획과 피드백으로 연결

KUBERNETES

조운호 · Platform Engineer

[github.com/CodingPenguin-yoon](#)
[code.penguin.yoon@gmail.com](#)
[yoonman.page](#)

HOW THE WORK EVOLVED

제어 대상과 성공 조건을 나눠 운영 자동화를 설계했습니다.

초기에는 VM 생성부터 애플리케이션 배포까지 한 흐름으로 묶으려 했습니다. 테스트 과정에서 두 작업의 권한과 실패 방식이 다르다는 것을 확인하고, 인프라 변경은 Gjallar로, 애플리케이션 배포는 Heimdall로 책임을 분리했습니다.

01 · LEARN

K-Le-PaaS

입력 → 계획 → 확인 → 실행 → 피드백으로 운영 요청을 구조화했습니다.



02 · SPLIT RESPONSIBILITY

Heimdall

애플리케이션 배포와 Preview 전환

Gjallar

Proxmox 관찰과 제한된 변경 통제

VALIDATION ENVIRONMENT

직접 운영하는 환경에서 확인합니다.

Proxmox 3노드와 IPFire로 분리한 RED·GREEN·ORANGE 네트워크, NAS/NFS, WireGuard·OCI reverse proxy 환경에서 정상 경로와 실패 경로를 검증합니다.

Proxmox VE 3-node

IPFire segmentation

NAS / NFS

WireGuard / OCI

CORE SKILLS

Platform	Linux · Docker · Kubernetes · Proxmox
Backend	Python · FastAPI · PostgreSQL · REST API
Delivery	Git · NGINX · GitHub Actions · Prometheus
Network	IPFire · WireGuard · Reverse Proxy · NFS

COMMON STANDARD

실행 전 대상을 고정하고, 실행 후 실제 상태를 확인하며, 불확실한 결과를 성공으로 바꾸지 않습니다.

01

HEIMDALL

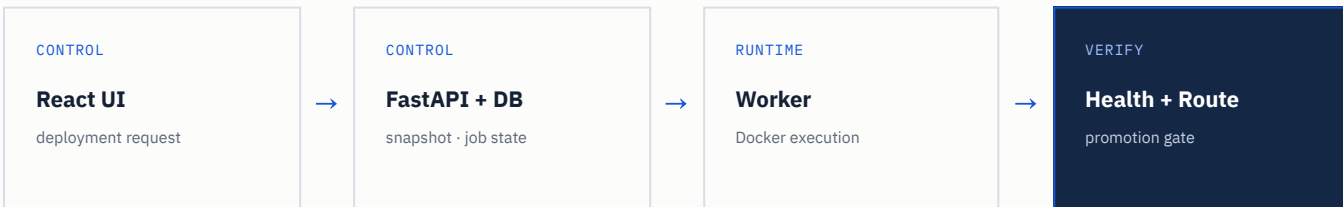
APPLICATION DELIVERY AUTOMATION · PERSONAL PROJECT

저장소 등록부터 Preview 전환까지 반복 배포 절차를 하나의 흐름으로 자동화했습니다.

공개 GitHub 저장소를 Docker Preview로 배포하는 self-hosted 플랫폼입니다. 애플리케이션마다 반복하던 빌드·실행·DB·공개 경로 준비를 공용 배포 흐름으로 묶었습니다.

PROBLEM	ROOT CAUSE	DECISION
애플리케이션마다 실행 환경과 공개 경로를 반복해서 준비해야 했습니다. 저장소 checkout과 Docker build·실행, 환경 설정, 프로젝트 DB, NGINX route를 애플리케이션마다 다시 연결하고 결과를 별도로 확인해야 했습니다.	배포에 필요한 설정과 작업 순서가 재사용 가능한 실행 단위로 구조화되지 않았습니다. commit·환경 설정·DB 연결·route 구성과 검증 절차가 개별 작업으로 흩어져 있어, 같은 배포 과정을 애플리케이션마다 반복해야 했습니다.	입력을 고정하면 candidate 준비부터 검증·전환까지 자동 실행되도록 만들었습니다. exact commit과 설정 snapshot을 기준으로 Docker candidate와 필요한 DB·route를 준비합니다. health와 route probe까지 통과하면 Preview를 전환하고, 실패하면 기존 정상 경로를 유지합니다.

BEFORE	AFTER
애플리케이션마다 VM·Docker·공개 경로를 수동 준비 새 버전의 실행 여부와 실제 서비스 응답을 별도로 확인	실행 중인 공용 플랫폼에 저장소와 설정을 등록 candidate 검증과 Preview 전환을 하나의 배포 흐름으로 수행



AUTOMATED DELIVERY WITH A SAFETY GATE

저장소 등록부터 Preview 전환까지 하나의 배포 작업으로 실행합니다.

배포 대상을 고정한 뒤 build·실행·health·route 검증과 전환을 순서대로 자동화했습니다. candidate 검증은 안전한 전환을 위한 조건입니다.

01	02	03	04	05
PIN	BUILD	CHECK	ROUTE	PROMOTE
exact SHA와 설정 snapshot 고정	분리된 Docker candidate 빌드	서비스별 health 확인	NGINX 적용 후 endpoint probe	검증된 Preview만 활성화

FAILURE INJECTION

의도적으로 backend 시작 실패를 넣었습니다.

3-tier 테스트 애플리케이션의 새 candidate가 중단되고, 이전 stable Preview의 응답이 계속 제공되는 것을 실제 배포로 확인했습니다.

NEW

candidate
failed

≠

ACTIVE

stable kept
serving

RESULT 01

실행 중인 플랫폼에서 3분 이내에 전환했습니다.

개인 홈페이지의 build·health·route 검증과 Preview 전환까지 확인했습니다. 이 시간에는 VM 생성이 포함되지 않습니다.

RESULT 02

프로젝트별 PostgreSQL도 함께 제공했습니다.

3-tier 테스트에서 프로젝트별 DB·role 생성과 password-file 기반 연결 정보 주입을 확인했습니다.

AUTOMATED FLOW

Public GitHub · fixed main · manual deploy · single Docker host · health and route gate

NEXT EXTENSIONS

VM provisioning · private Git · automatic webhook · TLS · multi-host · data rollback

[Repository ↗](#)[Gateway tests ↗](#)[Runtime smoke ↗](#)

02

GJALLAR

CONTROLLED INFRASTRUCTURE OPERATIONS · PERSONAL PROJECT

API 응답이 아니라 변경 후 실제 VM 상태로 성공을 판단합니다.

Proxmox actual inventory를 관찰하고, 제한된 변경을 사전 검증·승인·사후 확인 절차로 실행하는 운영 제어 도구입니다.

PROBLEM	RISK	DECISION
수동 VM 작업은 반복 입력과 사전 조건 누락에 취약했습니다. VM마다 CPU·메모리 사양과 IP를 다시 확인해야 했고, 중복 요청이나 IP 충돌을 작업 전에 일관되게 차단하기 어려웠습니다.	Proxmox가 요청을 수락해도 목표 상태에 도달했다고 단정할 수 없었습니다. 비동기 task의 timeout, 누락된 task reference, task 완료 후 실제 상태 불일치가 발생할 수 있으므로 API 응답과 운영 결과를 분리해야 했습니다.	관찰과 제한된 변경 통제에 제품 범위를 좁혔습니다. live inventory를 기준으로 권한 ·IP·network·storage를 확인하고, 승인된 작업만 실행합니다. 전체 VM lifecycle 자동화보다 안전한 실행 경계를 우선했습니다.

SOURCE OF TRUTH		OPERATION RECORD
Proxmox VE Node / VM inventory Task reference and status Observed resource state	READ LIVE STATE ↔ EXECUTE EXPLICIT CHANGE	Gjallar Operator intent and policy Approval · idempotency Post-check and reconciliation

BEFORE	AFTER
VM마다 사양·IP 조건을 다시 확인	Profile 재사용 + live preflight로 잘못된 요청 차단

GUARDED EXECUTION

변경 작업을 같은 버튼이 아니라, 같은 검증 원칙으로 묶었습니다.

작업별 위험은 다르지만 모든 변경은 계획·확인·실행·실제 상태 재조회로 끝납니다.

01 PLAN live inventory로 사전 조건 계산	02 CONFIRM 권한·승인·위험 확인	03 EXECUTE 제한된 Proxmox API 변경
04 TRACK UPID task 완료 상태 확인	05 POST-CHECK 변경 후 실제 VM 상태 조회	06 RECORD 성공 또는 재확인 상태 기록

01 CREATE VM Profile · IP preflight · 승인 · UPID · 생성 후 상태 확인	02 START 명시적 확인 · idempotency · task polling · running 확인
03 GRACEFUL SHUTDOWN 허용 상태 확인 · 제한된 실행 · 종료 상태 재조회	04 GUIDED UNLOCK 짧은 지침 발급 · 운영자 실행 · API로 lock 해제 확인

NON-SUCCESS PATH		
TIMEOUT 응답 지연을 자동 성공으로 변경하지 않음	MISSING TASK task reference가 없으면 결과를 확정하지 않음	STATE MISMATCH 예상 상태와 다른면 재확인 대상으로 기록

RESULT 반복 입력을 줄이고, 변경 결과를 실제 상태로 다시 확인했습니다. VM Profile로 자원 사양을 재사용하고 IP 충돌을 사전에 확인합니다. Create·Start·graceful Shutdown·제한된 guided unlock의 task와 after-state를 검증했습니다.	CONTROL RANGE 반복 작업은 자동화하고, 고위험 변경은 운영자 판단에 남겼습니다. Create·Start·graceful Shutdown과 제한된 guided unlock을 승인 흐름으로 제공하고, migration·snapshot·임의 shell 작업은 분리했습니다.
--	--

github.com/CodingPenguin-yeon/Gjallar

03

K-LE-PAAS

KUBERNETES OPERATIONS · TEAM OF 2 · 2025.09-12

모호한 자연어를 허용된 Kubernetes 실행 계획으로 바꿨습니다.

웹과 Slack의 운영 요청을 해석하고, 사용자가 확인할 수 있는 계획과 실행 결과·모니터링 피드백으로 연결한 팀 프로젝트입니다.

PROBLEM	SOLUTION	RESULT
자연어를 그대로 실행하면 해석 오류가 운영 변경으로 이어질 수 있습니다. 모호한 요청에서 대상과 의도를 분리하고, 시스템이 지원하는 작업과 인자만 실행하도록 제한할 필요가 있었습니다.	요청을 CommandRequest와 허용된 CommandPlan으로 구조화했습니다. 상태·로그 조회, 재시작·스케일링·버전 롤백 API로 연결하고 실행 결과와 외부 URL을 다시 사용자에게 반환했습니다.	실행 결과와 운영 정보를 다시 사용자에게 연결했습니다. Kubernetes 작업 결과에 더해 Prometheus 지표와 Ingress 외부 URL을 웹·Slack 요청 흐름으로 반환했습니다.



INITIAL OPTION	TEAM DECISION
온프레미스 배포 + NCP 게이트웨이 직접 운영 환경을 활용하는 구성을 검토	NCP SourcePipeline → NKS 공모전 조건과 구축 범위를 비교해 NCP 배포 흐름으로 합의

FROM REQUEST TO FEEDBACK

요청을 실행하고, 상태·URL·지표를 다시 사용자에게 돌려줬습니다.

자연어를 자유 형식 명령으로 실행하지 않고, 검증 가능한 작업 단위로 변환해 Kubernetes API와 NKS 관측 경로에 연결했습니다.

01	02	03	04	05
PARSE	VALIDATE	PLAN	EXECUTE	FEEDBACK
자연어를 command-parameters JSON으로 변환	CommandRequest로 형식과 대상 검증	허용된 CommandPlan(tool, args) 생성	Kubernetes Python Client API 호출	상태·URL·메트릭을 사용자에게 반환

MY CONTRIBUTION

자연어 실행 계획

command-parameters 구조화와 Kubernetes 실행·조회 연결

NKS 모니터링

Prometheus 설치·수집 대상 구성, CPU·메모리·디스크·네트워크 조회 API

서비스 접근 경로

Ingress 외부 주소와 사용자·저장소별 service URL 생성·저장·조회

MERGED EVIDENCE

PR #28 NLP / Kubernetes operations

PR #42 NKS monitoring API

PR #63 Deployment URL

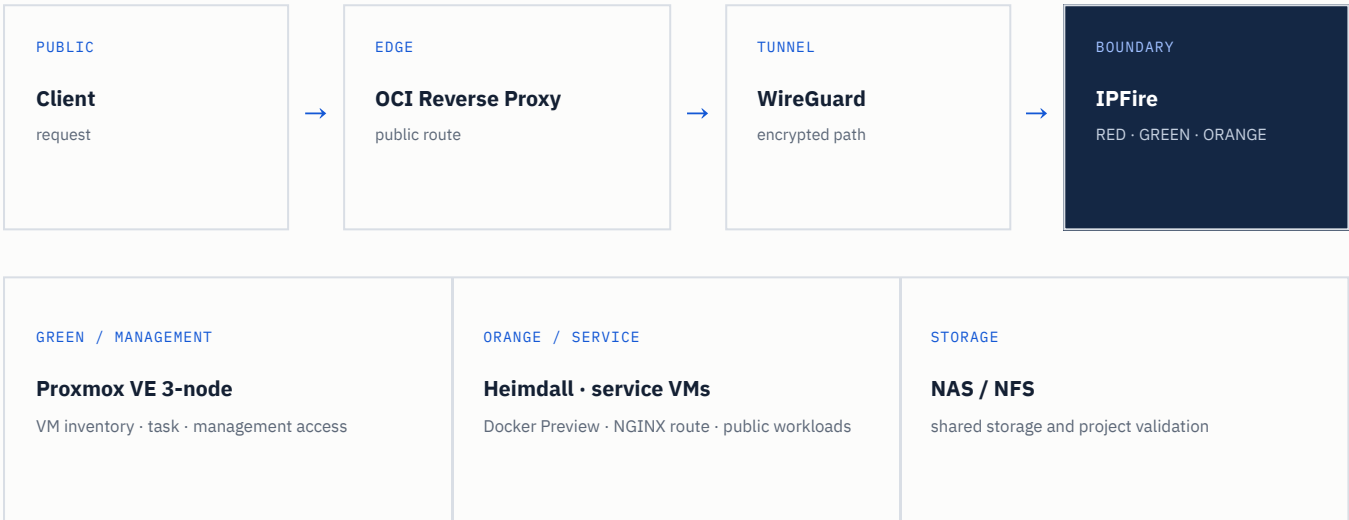
RESTART TARGET	API ACTION	AFTER STATE
owner / repository → Deployment	patch_namespaced_deployment()	state · metrics · service URL
저장소 식별자를 실행 대상 namespace와 Deployment로 매핑	pod template annotation을 변경해 rollout restart 수행	실행 이후 확인에 필요한 운영 정보를 요청 흐름으로 반환

DELIVERY RECORD	RUNTIME RESULT	DEMO
Backend·Frontend authored merged PR 48개	NKS 작업·모니터링·Ingress URL 흐름 시연	12-minute walkthrough ↗

WHERE I VERIFY

직접 운영하는 환경에서 정상 경로와 실패 경로를 확인합니다.

기능 구현을 API 응답에서 끝내지 않고, 네트워크 경로와 런타임·자원 상태까지 확인할 수 있는 환경을 유지합니다.



WHAT THESE PROJECTS SHOW

- 01

배포

반복되던 build·DB·route 준비와 Preview 전환을 하나의 배포 흐름으로 자동화했습니다.
- 02

인프라 변경

권한·중복 방지·task·actual state 확인으로 제한된 변경을 통제했습니다.
- 03

운영 인터페이스

모호한 자연어를 허용된 계획으로 바꾸고 결과와 지표를 다시 연결했습니다.

EDUCATION & CERTIFICATIONS

광운대학교 전자통신공학과 · 2026.02 졸업

정보처리기사 · 리눅스마스터 2급

CONTACT

조윤호 · Platform Engineer

code.penguin.yoon@gmail.com
github.com/CodingPenguin-yoon
yoonman.page